Eesha Chavan D20A/15

Blockchain Lab Exp 1

AIM: Cryptography in Blockchain, Merkle root Tree Hash

Theory:

1. Cryptographic Hash functions in Blockchain

A **cryptographic hash function** is a mathematical algorithm that converts input data of any size into a fixed-length output (called a *hash* or *digest*).In blockchain, the most commonly used hash function is **SHA-256**.

Properties of Cryptographic Hash Functions

Deterministic – Same input always produces the same hash.
Fixed Output Size – No matter the input size, output length is fixed.
Pre-image Resistance – Cannot reverse the hash to find original input.
Collision Resistance – Hard to find two different inputs with the same hash.
Avalanche Effect – Small input change → completely different hash.

2. What is a Merkle Tree?

A Merkle Tree (Hash Tree) is a tree structure where:

- Each leaf node is a hash of a transaction.
- Each non-leaf node is a hash of its two child hashes.
- The top hash is called the Merkle Root.

It is used to efficiently verify data integrity in blockchain.

3. What is a Cryptographic Puzzle and explain the Golden Nonce

A cryptographic puzzle is a mathematical problem that:

- Is hard to solve
- Is easy to verify

In blockchain, this is used in Proof-of-Work (PoW).

Miners must find a number called a nonce such that:

```
Hash(Block Data + Nonce) → starts with required number of zeros
```

Example:

```
0000ab34f567...
```

The difficulty increases as required leading zeros increase.

What is the Golden Nonce?

The Golden Nonce is the special nonce value that solves the cryptographic puzzle.

It is the number that makes the block hash satisfy the required difficulty condition.

Example:

```
Block data + 45231 → hash starts with 00000
```

Here, `45231` is the Golden Nonce.

Miners try millions of nonces until they find this one.

4. How does a Merkle Tree work?

Let's say we have 4 transactions: `T1, T2, T3, T4`

Step 1: Hash all transactions

```
H1          =
hash(T1)   H2
=    hash(T2)
H3          =
hash(T3)   H4
= hash(T4)
```

Step 2: Combine pairwise and hash again

```
H12 = hash(H1 + H2)
H34 = hash(H3 + H4)
```

Step 3: Final combination

```
Merkle Root = hash(H12 + H34)
```

If the number of transactions is odd → duplicate the last transaction.

This final hash (Merkle Root) is stored inside the block header.

5. Benefits of Merkle Tree

1. Efficient Verification: You don't need all transactions to verify one transaction.

2. Saves Memory: Lightweight nodes (SPV nodes) only store block headers.

3. Fast Detection of Tampering: If one transaction changes → Merkle Root changes.

4. Scalability: Efficient even with thousands of transactions.

6.Use cases of Merkle Tree

1. Blockchain (Bitcoin, Ethereum):Used to store transaction hashes.

2. Distributed Systems:Used in IPFS and distributed storage verification.

3. Git Version Control:Git uses Merkle Trees to track file changes.

4. Data Integrity Verification:Used to verify large datasets efficiently.

5. Peer-to-Peer Networks:Ensures file chunks are not modified.

Codes:

1.Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib
def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
    return hash_string
# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)#error statement
```

```
Enter a string: eesha
Hash: 20970482291b8b7f0f6275650b8d80974d9b2211cda7b89b103a124ed26d7f46
```

2. Program to generate required target hash with input string and nonce

```python
import hashlib


# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

```
Enter a string: eeshachavan
Enter the nonce: 17
Hash Code: 569c86a5a9eeeb7b108a7602354e2f54280a465d7ce5d039834e81b44b38e01f
```

3. Python code for Solving Puzzle for leading zeros with expected nonce and given string

```python
import hashlib

def find_nonce(input_string, num_zeros):
  nonce = 0
  hash_prefix = '0' * num_zeros

  while True:

  # Concatenate the string and nonce
    hash_string = input_string + str(nonce)

  # Calculate the hash using SHA-256
    hash_object = hashlib.sha256(hash_string.encode('utf-8'))
    hash_code = hash_object.hexdigest()

  # Check if the hash code has the required number of leading zeros
    if hash_code.startswith(hash_prefix):
      print("Hash:", hash_code)
      return nonce

    nonce += 1

# Get user input
input_string = "B"
num_zeros = 2

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)


# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
Hash: 00ceeab4df7002aad9c625c69e81c47d07f71027ad3600787a73c7e91fdf8d61
Input String: B
Leading Zeros: 2
Expected Nonce: 18
```

4. Generating Merkle Tree for given set of Transactions

```python
import hashlib

def build_merkle_tree(transactions):

    if len(transactions) == 0:
        return None

    # Step 1: Hash all transactions first
    print("Initial Transaction Hashes:")
    transactions = [
        hashlib.sha256(tx.encode('utf-8')).hexdigest()
        for tx in transactions
    ]

    for h in transactions:
        print(h)
    print()

    # Step 2: Build the Merkle Tree
    level = 1
    while len(transactions) > 1:
        print(f"Level {level} Hashes:")

        # If odd number of transactions, duplicate last one
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i + 1]
            hash_combined = hashlib.sha256(
                combined.encode('utf-8')
            ).hexdigest()
            print(hash_combined)
            new_transactions.append(hash_combined)
```

```python
        print()
        transactions = new_transactions
        level += 1

    return transactions[0]


# Example usage with valid transactions
transactions = [
    "Eesha -> Varun : $100",
    "Anish -> Parth : $500",
    "Gourish -> Atharva : $200",
    "Aditya -> Soniya : $400",
    "Eesha -> Pranav : $1000"
]

merkle_root = build_merkle_tree(transactions)
print("Final Merkle Root:", merkle_root)
```

```
Initial Transaction Hashes:
35ea2b67986230d3eb6d2f28d0faca4c11e25038a0589facad5074d929ae4f31
37c850dbef94bc6954229ee137df618ef56a3d235b43990a140b4f9688e1d635
ff76705fc1226a92f042e5815b9a86c3c4cf80b5d9c9392f02c378a18a23768c
3fe833b3ad5a5182f89830dcde0a36f2fe455c80c3dec1ffe89c87ae4f612a28
36abfd92062e2971eaebbfdcded1a7728b9bd7cd12b69843d9652ac03448756b

Level 1 Hashes:
bbe6d83d99125cbb9135a11d147673ed2c6f4f730c41c176282dc10390c869b1
c9f5a5a8c1badcd6eb1d1727d5e8dd60a5bc5f7fb86eacc356b0a4a7d93fc4d9
4290426ea79b83744b7c227fb5440cff3d15fa891c2273bac0a612c86850d6f7

Level 2 Hashes:
dd9d2220a6c02395835149a82748e126b103d6af0d77031c77ca0f0b0ec80de0
4037715de78ed0b5206f18d650f4820b40d05cedb96acafcb1eedace7ba6aeb9

Level 3 Hashes:
246f51235c7b41273fe111d9de92d667b00ffed1db14c27348e3af356d48ab70

Final Merkle Root: 246f51235c7b41273fe111d9de92d667b00ffed1db14c27348e3af356d48ab70
```

Conclusion:

Cryptographic hash functions secure blockchain data by ensuring integrity and immutability. Merkle Trees efficiently organize and verify large numbers of transactions using a single Merkle Root.Cryptographic puzzles in Proof-of-Work maintain network security by requiring computational effort.Together, these mechanisms make blockchain systems secure, transparent, and tamper-resistant.